

Iterative solution of linear systems

April 16, 2026

Justification

As an alternative to Gaussian elimination, iterative methods can be an efficient way to solve the linear system from PDEs. We discuss basic iterative methods and the notion of preconditioning.

Two different approaches

Solve $Ax = b$

Direct methods:

- Deterministic
- Exact up to machine precision
- Expensive (in time and space)

Iterative methods:

- Only approximate
- Cheaper in space and (possibly) time
- Convergence not guaranteed

0.1. Stationary iteration

Abstract presentation

- To solve $Ax = b$; too expensive; suppose $K \approx A$ and solving $Kx = b$ is possible
- Define $Kx_0 = b$, then error correction $x_0 = x + e_0$, and $A(x_0 - e_0) = b$
- so $Ae_0 = Ax_0 - b = r_0$; this is again unsolvable, so
- $K\tilde{e}_0 = r_0$ and $x_1 = x_0 - \tilde{e}_0$.
- In one formula:

$$x_1 = x_0 - K^{-1}r_0$$

- now iterate: $e_1 = x_1 - x$, $Ae_1 = Ax_1 - b = r_1$ et cetera

Iterative scheme:

$$x_{i+1} = x_i - K^{-1}r_i \quad \text{where } r_i = Ax_i - b$$

Takeaway

Each iteration involves:

- multiplying by A ,
- solving with K

0.2. Polynomial iterative methods

Derivation by hand-waving

- Remember iteration:

$$x_1 = x_0 - K^{-1}r_0, \quad r_0 = Ax_0 - b$$

and conclusion

$$r_n = (I - AK^{-1})^n r_0$$

- Abstract relation between true solution and approximation:

$$x_{\text{true}} = x_{\text{initial}} + K^{-1}\pi(AK^{-1})r_{\text{initial}}$$

- Cayley-Hamilton theorem implies

$$(K^{-1}A)^{-1} = -\pi(K^{-1}A)$$

- inspires us to scheme:

$$x_{i+1} = x_0 + K^{-1}\pi^{(i)}(AK^{-1})r_0$$

Sequence of polynomials of increasing degree

Residuals

$$x_{i+1} = x_0 + K^{-1}\pi^{(i)}(AK^{-1})r_0$$

Multiply by A and subtract b :

$$r_{i+1} = r_0 + \tilde{\pi}^{(i)}(AK^{-1})r_0$$

So:

$$r_i = \hat{\pi}^{(i)}(AK^{-1})r_0$$

where $\hat{\pi}^{(i)}$ is a polynomial of degree i with $\hat{\pi}^{(i)}(0) = 1$.

$\Rightarrow$ convergence theory

Computational form

$$x_{i+1} = x_0 + \sum_{j \leq i} K^{-1} r_j \alpha_{ji}.$$

or equivalently:

$$x_{i+1} = x_i + \sum_{j \leq i} K^{-1} r_j \alpha_{ji}.$$

or:

$$\begin{cases} r_i = Ax_i - b \\ x_{i+1} \gamma_{i+1,i} = K^{-1} r_i + \sum_{j \leq i} x_j \gamma_{ji} \\ r_{i+1} \gamma_{i+1,i} = AK^{-1} r_i + \sum_{j \leq i} r_j \gamma_{ji} \end{cases}$$

Takeaway

Each iteration involves:

- multiplying by A ,
- solving with K

Takeaway

Each iteration involves:

- multiplying by A ,
- solving with K
- inner products!

Symmetric Positive Definite case

Three term recurrence is enough:

$$\begin{cases} x_{i+1} = x_i - \delta_i p_i \\ r_{i+1} = r_i - \delta_i A p_i \\ p_{i+1} = K^{-1} r_{i+1} + \gamma_i p_i \end{cases}$$

$$\gamma_i = \frac{r_{i+1}^t r_{i+1}}{r_i^t r_i}, \delta_i = \frac{r_i^t r_i}{p_i^t A p_i}$$

Preconditioned Conjugate Gradients

Compute $r^{(0)} = Ax^{(0)} - b$ for some initial guess $x^{(0)}$

for $i = 1, 2, \dots$

solve $Kz^{(i-1)} = r^{(i-1)}$

$\rho_{i-1} = r^{(i-1)T} z^{(i-1)}$

if $i = 1$

$p^{(1)} = z^{(0)}$

else

$\beta_{i-1} = \rho_{i-1} / \rho_{i-2}$

$p^{(i)} = z^{(i-1)} + \beta_{i-1} p^{(i-1)}$

endif

$q^{(i)} = Ap^{(i)}$

$\delta_i = \rho_{i-1} / p^{(i)T} q^{(i)}$

$x^{(i)} = x^{(i-1)} - \delta_i p^{(i)}$

$r^{(i)} = r^{(i-1)} - \delta_i q^{(i)}$

check convergence; continue if necessary

takeaway

Each iteration involves:

- Matrix-vector product
- Preconditioner solve
- Two inner products
- Other vector operations.

Three popular iterative methods

- Conjugate gradients: constant storage and inner products; works only for symmetric systems
- GMRES (like FOM): growing storage and inner products: restarting and numerical cleverness
- BiCGstab and QMR: relax the orthogonality

0.3. Let's go parallel

0.4. Computational aspects of iterative methods

What's in an iterative method?

From easy to hard

- Vector updates
These are trivial
- Inner product
- Matrix-vector product
- Preconditioner solve

Inner products: collectives

Collective operation: data from all processes is combined.
(Is a matrix-vector product a collective?)

Examples: sum-reduction, broadcast

These are each other's mirror image, computationally.

Naive realization of collectives

Broadcast:



Single message:

$$\alpha = \text{message startup} \approx 10^{-6}s, \quad \beta = \text{time per word} \approx 10^{-9}s$$

- Time for message of n words:

$$\alpha + \beta n$$

Naive realization of collectives

Broadcast:



Single message:

$$\alpha = \text{message startup} \approx 10^{-6}s, \quad \beta = \text{time per word} \approx 10^{-9}s$$

- Time for message of n words:

$$\alpha + \beta n$$

- Single inner product: $n = 1$

Naive realization of collectives

Broadcast:



Single message:

$$\alpha = \text{message startup} \approx 10^{-6}s, \quad \beta = \text{time per word} \approx 10^{-9}s$$

- Time for message of n words:

$$\alpha + \beta n$$

- Single inner product: $n = 1$
- Time for collective?

Naive realization of collectives

Broadcast:



Single message:

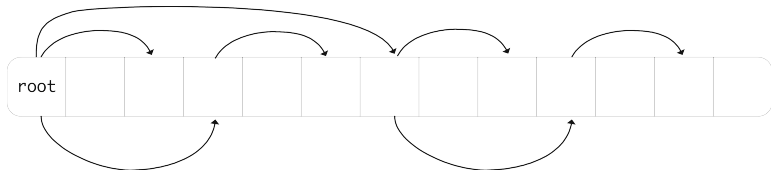
$$\alpha = \text{message startup} \approx 10^{-6}s, \quad \beta = \text{time per word} \approx 10^{-9}s$$

- Time for message of n words:

$$\alpha + \beta n$$

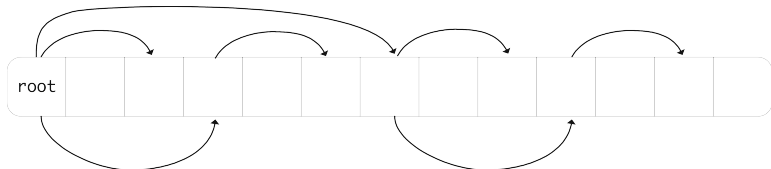
- Single inner product: $n = 1$
- Time for collective?
- Can you improve on that?

Better implementation of collectives



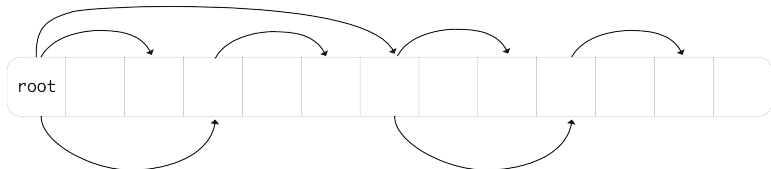
- What is the running time now?

Better implementation of collectives



- What is the running time now?
- Can you come up with lower bounds on the α, β terms? Are these achieved here?

Better implementation of collectives

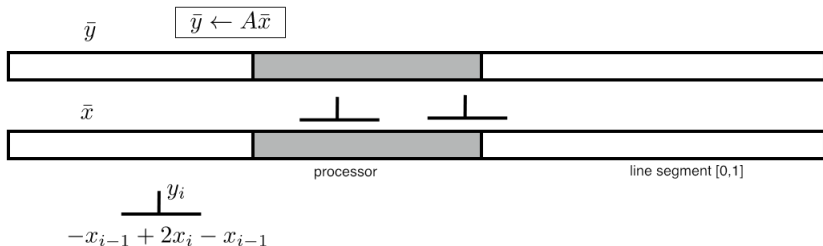


- What is the running time now?
- Can you come up with lower bounds on the α, β terms? Are these achieved here?
- How about the case of really long buffers?

Matrix-vector product

Parallelization

Assume each process has the matrix values and vector values in part of the domain.



Processor needs to get values from neighbors.

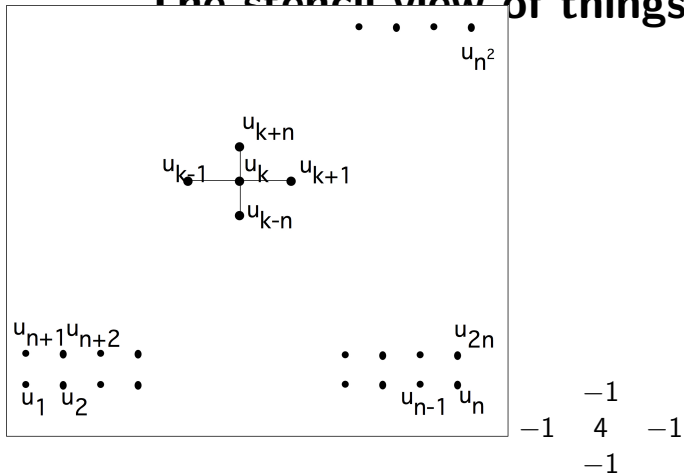
Second order PDEs; 2D case

$$\begin{cases} -u_{xx}(\bar{x}) - u_{yy}(\bar{x}) = f(\bar{x}) & x \in \Omega = [0, 1]^2 \\ u(\bar{x}) = u_0 & \bar{x} \in \delta\Omega \end{cases}$$

Now using central differences in both x and y directions:

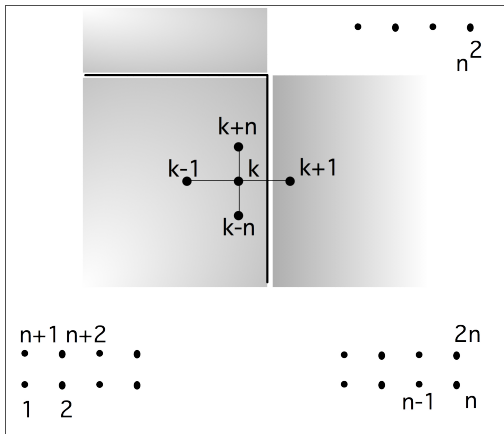
$$4u(x, y) - u(x + h, y) - u(x - h, y) - u(x, y + h) - u(x, y - h)$$

The stencil view of things



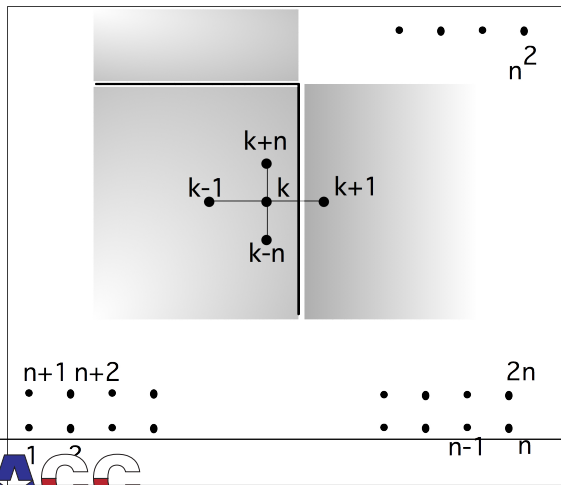
Operator view of spmvp

Difference stencil



PDE, 2D case

A difference stencil applied to a two-dimensional square domain, distributed over processors. Each point connects to neighbors $\Rightarrow$ each process connects to neighbors.



Matrices in parallel

$$y \leftarrow Ax$$

and A, x, y all distributed:

